

TP - ANDROID - Lab 2 : Découverte de l'environnement et structure d'un projet Android

Objectif du TP

À la fin de ce lab, vous serez capable de :

- Comprendre les différentes parties de l'interface Android Studio
- Naviguer dans un projet Android
- Comprendre la structure d'un projet Android
- Créer et exécuter une application simple (Hello World)
- Comprendre le rôle des fichiers principaux (Activity, XML, Manifest)

Prérequis

- Android Studio installé et fonctionnel
- Un projet Android créé (Lab 1)

1. Présentation de l'environnement de développement

Android Studio est l'IDE officiel pour le développement Android. Il fournit plusieurs outils essentiels.

1.1 Gestionnaire de projet (Project Explorer)

Situé à gauche de l'écran.

Permet de :

- Naviguer dans les fichiers du projet
- Organiser le code
- Accéder rapidement aux ressources



Astuce :

- Utiliser la vue **Android** (plus simple) plutôt que **Project**

1.2 Éditeur de code

Zone centrale de l'IDE.

Permet de :

- Écrire du code Java

- Modifier les fichiers XML
- Bénéficier de l'auto-complétion
- Voir les erreurs en temps réel

1.3 Logcat

Outil essentiel pour le debug.

Permet de :

- Voir les logs de l'application
- Identifier les erreurs (crash, exceptions)
- Filtrer les logs par application

Accès :

- En bas de l'écran → onglet **Logcat**

2. Structure d'un projet Android

Dans Android Studio, un projet est organisé en plusieurs dossiers importants.

2.1 AndroidManifest.xml

Chemin : app > manifests > AndroidManifest.xml

Rôle : C'est un fichier essentiel qui.

- Déclare les composants de l'application (Activity, Service, etc.)
- Définit les permissions
- Configure le comportement global de l'application

Exemple :

```
1 <manifest ...>
2   <application
3     android:label="MyFirstApp">
4
5     <activity android:name=".MainActivity">
6       <intent-filter>
7         <action android:name="android.intent.action.MAIN" />
8         <category
9 android:name="android.intent.category.LAUNCHER" />
10        </intent-filter>
11      </activity>
12    </application>
13  </manifest>
```

Explication :

- MainActivity est le point d'entrée de l'application
- LAUNCHER indique que c'est l'écran principal

2.2 Dossier java

Chemin : app > java > votre.package

Rôle : Contient le code source Java de l'application.

Exemple :

- MainActivity.java

C'est ici que vous implémentez :

- La logique métier
- Les interactions utilisateur

2.3 Dossier res (resources)

Chemin : app > res

Rôle : Contient toutes les ressources non-code.

Layouts (res/layout)

- Contiennent les interfaces graphiques (XML)
- Exemple : activity_main.xml

Values (res/values)

- Contiennent des valeurs statiques :
 - strings.xml (textes)
 - colors.xml
 - themes.xml

Drawable (res/drawable)

- Images, icônes, formes

3. Création d'un Hello World

 Si ce n'est pas déjà fait :

1. Créer un projet **Empty Activity**
2. Nom : HelloWorldApp

3. Langage : **Java**

3.1 Lancer l'application

1. Cliquer sur **Run**
2. Choisir un émulateur
3. Vérifier que l'application démarre

3.2 Résultat attendu

Un écran avec le texte :

```
1 | Hello World!
```

4. Explication de MainActivity.java

Fichier : MainActivity.java

Exemple simplifié :

```
1 | public class MainActivity extends AppCompatActivity {  
2 |  
3 |     @Override  
4 |     protected void onCreate(Bundle savedInstanceState) {  
5 |         super.onCreate(savedInstanceState);  
6 |         setContentView(R.layout.activity_main);  
7 |     }  
8 | }
```

Explication détaillée

onCreate()

- Méthode du cycle de vie de l'activité
- Appelée lors de la création de l'écran

savedInstanceState

- Contient l'état sauvegardé de l'activité
- Utilisé si l'application est recrée (rotation écran, manque mémoire)

super.onCreate(savedInstanceState)

- Appelle la logique de la classe parent
- Obligatoire pour le bon fonctionnement

setContentView(...)

- Définit l'interface graphique à afficher

- Ici : activity_main.xml

5. Explication de activity_main.xml

Fichier : res/layout/activity_main.xml

Exemple :

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <androidx.constraintlayout.widget.ConstraintLayout
3     xmlns:android="http://schemas.android.com/apk/res/android"
4     xmlns:app="http://schemas.android.com/apk/res-auto"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent">
7
8     <TextView
9         android:id="@+id/textView"
10        android:text="Hello World!"
11        android:layout_width="wrap_content"
12        android:layout_height="wrap_content"
13        app:layout_constraintTop_toTopOf="parent"
14        app:layout_constraintBottom_toBottomOf="parent"
15        app:layout_constraintStart_toStartOf="parent"
16        app:layout_constraintEnd_toEndOf="parent"/>
17
18 </androidx.constraintlayout.widget.ConstraintLayout>
```

Explication

ConstraintLayout

- Conteneur principal
- Permet de positionner les éléments

TextView

- Composant d’affichage de texte
- Affiche “Hello World!”

Contraintes

- Permettent de centrer le texte à l’écran

6. Modification simple (exercice)

Objectif : Modifier le texte affiché

1. Ouvrir activity_main.xml
2. Modifier :

```
1 android:text="Bienvenue dans mon app Android"
```

3. Relancer l’application

7. Vérification finale

À la fin du TP, vous devez :

- Comprendre l'interface Android Studio
- Savoir naviguer dans un projet
- Comprendre le rôle du Manifest, Java et res
- Comprendre une Activity
- Modifier une interface simple
- Lancer une application